

COMP 370 Mini Project 2: Pattern-Based Refactoring Report

300222668 Farhan Zariwala

1. Original Design Limitations

In Mini Project 1, the Server Redundancy Management System (SRMS) functioned correctly but lacked architectural flexibility and safety. The design review revealed three primary limitations:

- **Unrestricted Instantiation:** The MonitorMain class used a public constructor, meaning multiple instances of the monitor could accidentally be created, leading to split-brain scenarios or port binding conflicts.
- **Tight Coupling in Notifications:** The Monitor was directly responsible for logging internal state changes (like primary elections). If we wanted to add an email alert system or an external dashboard later, we would have had to modify the core MonitorMain code, violating the Open/Closed Principle.
- **Mixed Responsibilities:** ServerProcess handled both its base configuration (ID, ports, host) and its operational execution logic in a single file, making it harder to cleanly extend into distinct PrimaryServer and BackupServer types in the future without duplicating code.

2. Rationale and Pattern Applications

2.1 Singleton Pattern

Rationale: The Monitor acts as the central authority for heartbeat tracking and primary election. Only one active instance must exist per system. By implementing the Singleton pattern, we enforce a single, globally accessible instance.

Code Snippet:

```
13     private static MonitorMain instance;  
  
24     private MonitorMain(int port) { this.port = port; }
```

```

45     public static synchronized MonitorMain getInstance(int port) {
46         if (instance == null) {
47             instance = new MonitorMain(port);
48         }
49         return instance;
50     }

```

2.2 Abstraction–Occurrence Pattern

Rationale: To avoid duplicating basic server attributes in future extensions, we separated the common fields (the "abstraction") from the concrete execution logic (the "occurrence"). We created an abstract `ServerType` class to hold the configuration.

Code Snippet:

```

1  package server;
2
3  public abstract class ServerType {
4      protected final int id;
5      protected final int port;
6      protected final String monitorHost;
7      protected final int monitorPort;
8
9      public ServerType(int id, int port, String monitorHost, int monitorPort) {
10         this.id = id;
11         this.port = port;
12         this.monitorHost = monitorHost;
13         this.monitorPort = monitorPort;
14     }
15
16     public abstract void start() throws Exception;
17 }

```

```

public class ServerProcess extends ServerType {
    private volatile boolean running = true;
    private volatile boolean isPrimary = false;
    private final File logFile;

    public ServerProcess (int id, int port, String monitorHost, int monitorPort) {
        super(id, port, monitorHost, monitorPort);
        new File(pathname: "logs").mkdirs();
        this.logFile = new File("logs/server-" + id + ".log");
    }
}

```

2.3 Observer Pattern

Rationale: We needed to decouple the Monitor's core logic from the actions triggered by state changes (e.g., electing a new primary). We introduced an Observer interface, allowing multiple components to "listen" to the Monitor without the Monitor knowing their specific implementations.

Code Snippet:

```
19     private final List<Observer> observers = new ArrayList<>();
20     public void addObserver(Observer o) { observers.add(o); }
21     private void notifyObservers(String event) {
22         for (Observer o : observers) { o.update(event); }
23     }

110     notifyObservers("NEW_PRIMARY:" + primaryId);
```

3. Evaluation Results

The goal of this evaluation is to measure and compare the quality of the original SRMS against the refactored, pattern-based version.

Metric	Original (MP1)	Refactored (MP2)	Difference & Impact
Files/Classes	5 core classes	7 core classes	Increased due to ServerType, Observer, and EventLogger
Lines of Code (Server)	~140 LOC in ServerProcess	~20 LOC in ServerType + ~120 LOC in ServerProcess	Separation of concerns improved. Same overall size, but split logically.
Monitor Coupling	Tightly coupled to logging	Loosely coupled via Observer list	Improved: The Monitor no longer cares <i>how</i> events are processed, only that observers are notified.
Monitor Coupling	0 checks (Public constructor)	1 access point (getInstance)	Improved: Global consistency is guaranteed.

Interpretation and Justification

The application of the **Abstraction–Occurrence** pattern removed duplicated server attributes from being hardcoded in operational classes, improving maintainability.

Using **Observer** decoupled the Monitor from specific notification types, lowering coupling and making it easier to add new event listeners (like a GUI or Database logger) without modifying the Monitor class. While the **Singleton** pattern added a slight bit of boilerplate code to the Monitor, the trade-off is completely justified by the guarantee of system consistency.

4. Lessons Learned

This project demonstrated that applying design patterns often requires adding more files and slight structural complexity upfront, but it pays off by making the system significantly easier to safely modify and extend later. Software engineering is not just about making the code work, but structuring it for future maintainability.

5. Team Contributions

- **Farhan Zariwala 300222668:** Conducted UML review, implemented Singleton in MonitorMain, applied Abstraction-Occurrence to ServerProcess, implemented Observer pattern, generated updated PlantUML diagrams, and wrote the comparative evaluation report.